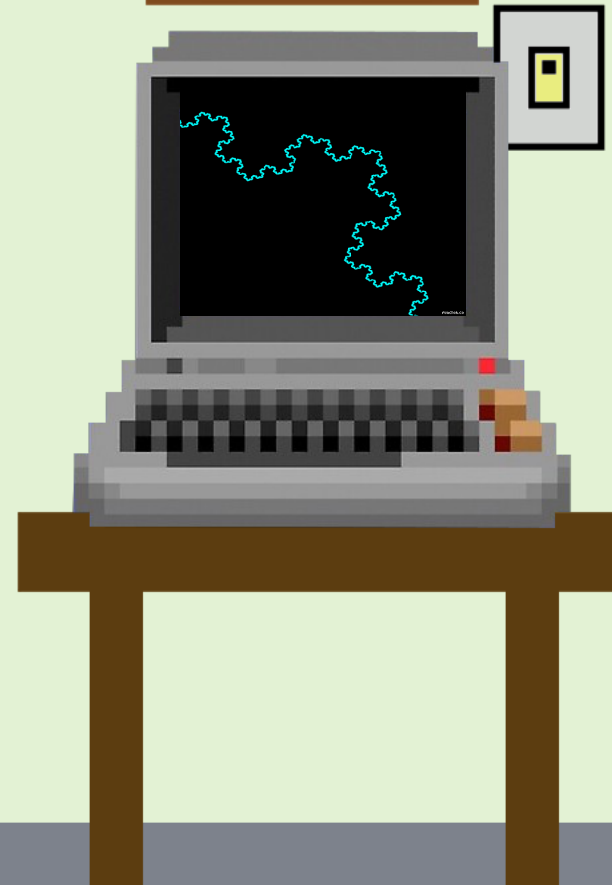


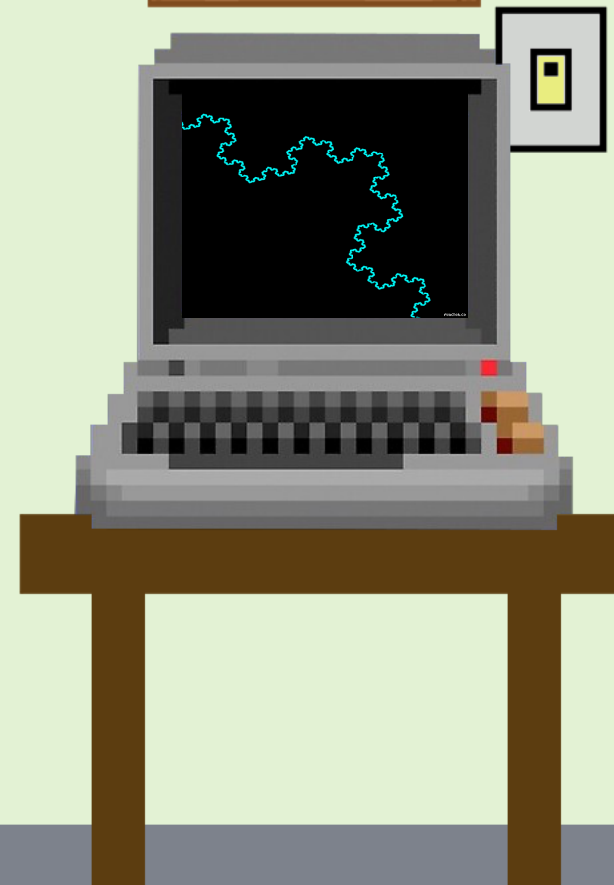
Tipos Estruturados

Programação Funcional



Tipos Estruturados

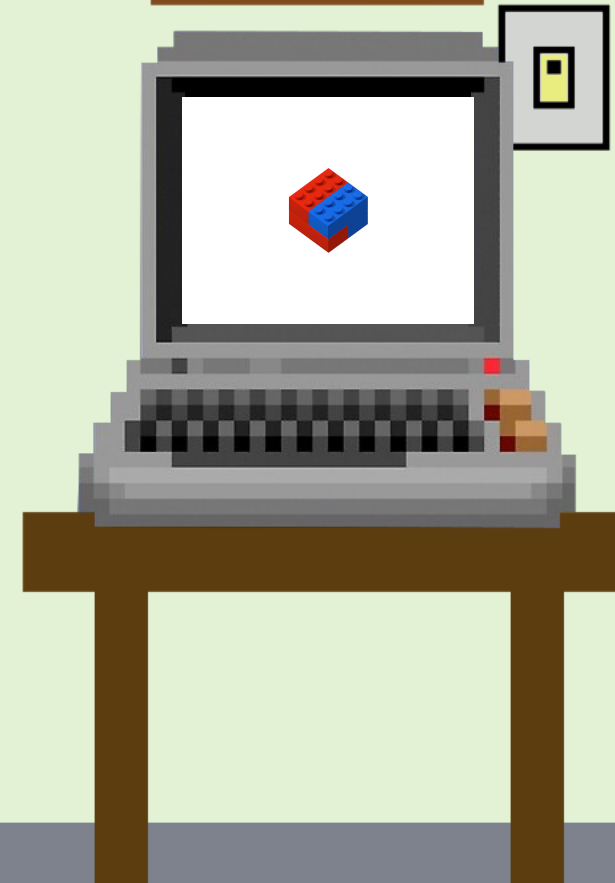
- Haskell admite a possibilidade de o usuário construir seus próprios tipos de dados
 - A partir de outros tipos primitivos ou estruturados
- Permite um grau muito maior de abstração dos problemas a serem resolvidos
- Tipos Estruturados
 - Produto cartesiano (Tuplas)
 - Listas



Tuplas

- Produto cartesiano
 - Podem ser duplas, triplas, n-uplas
 - Registros ou estruturas (programação imperativa)
- Coleções possivelmente heterogêneas de dados
 - Tamanho fixo
- Em Haskell, o tipo $(t_1, t_2, \dots, t_n)$ consiste de n-uplas de valores $(v_1, v_2, \dots, v_n)$ onde $v_1:t_1, v_2:t_2, \dots, v_n:t_n$

```
type Pessoa = (String, String, Int)
cliente :: Pessoa
cliente = ("Alberto Santos", "3521-0000", 27)
```



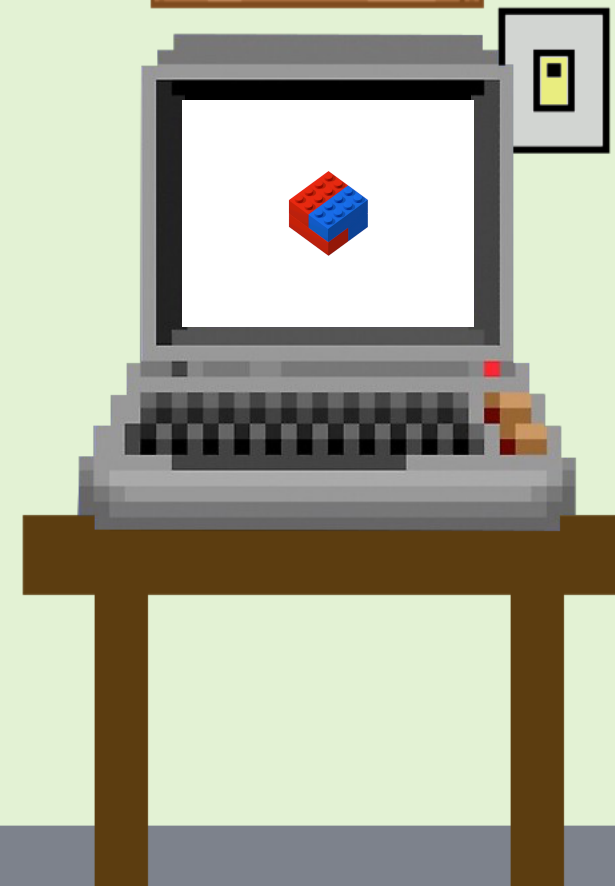
Tuplas

- Funções que manipulam tuplas são normalmente definidas utilizando casamento de padrões

```
nome :: Pessoa -> String  
nome (n, p, a) = n
```

```
fone :: Pessoa -> String  
fone (n, p, a) = p
```

```
anoNasc :: Pessoa -> Int  
anoNasc (n, p, a) = 2025-a
```



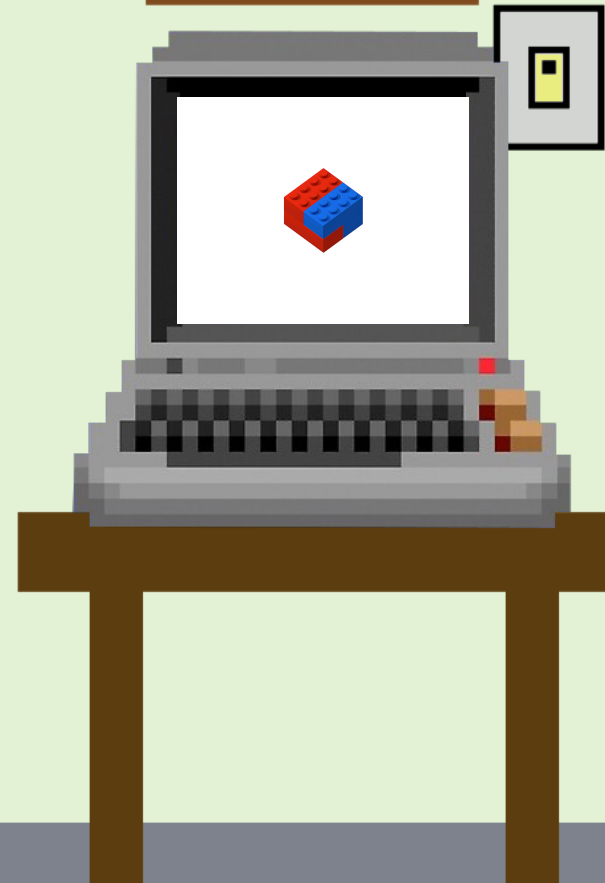
Tuplas

- Atenção aos tipos

```
somaPar :: (Int, Int) -> Int  
somaPar (a, b) = a + b
```

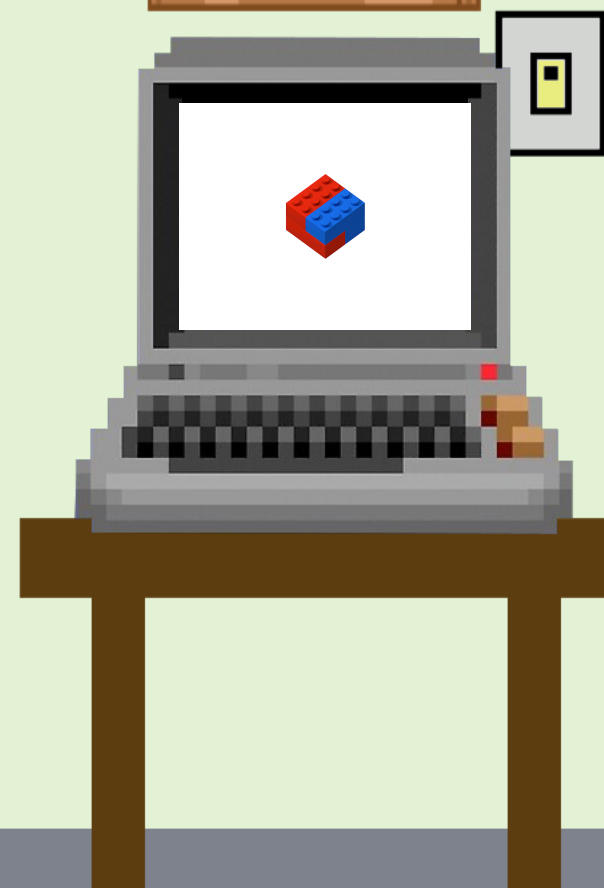
≠

```
somaPar :: Int -> Int -> Int  
somaPar a b = a + b
```



Exemplo

- Para o tipo `Data=(Int,Int,Int)`
- Definir uma função que verifica se a data é válida
 - `valida :: Data -> Bool`
- Definir uma função que verifica se o ano é bissexto
 - `bissexto :: Data -> Bool`
 - Múltiplos de 4 (exceto anos múltiplos de 100 que não são múltiplos de 400)



Exercício

- Dado o tipo Hora = (Int,Int,Int), composto por horas, minutos e segundos, crie as seguintes funções:
 - Recebe uma Hora e informa se é válida
 - Recebe uma Hora e retorna a quantidade de segundos desde a hora zero
 - Recebe um valor inteiro representando segundos e converte na hora correspondente
 - Recebe dois argumentos do tipo hora e retorna a diferença de tempo entre as duas

